

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Grafové databáze – přehled a srovnání



2023

Vedoucí práce:
Mgr. Tomáš Urbanec

Jan Gaja

Studijní program: Informatika,
Specializace: Obecná informatika

Bibliografické údaje

Autor: Jan Gaja
Název práce: Grafové databáze – přehled a srovnání
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2023
Studijní program: Informatika, Specializace: Obecná informatika
Vedoucí práce: Mgr. Tomáš Urbanec
Počet stran: 19
Přílohy: 1 CD/DVD
Jazyk práce: český

Bibliographic info

Author: Jan Gaja
Title: Graph databases – overview and comparison
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2023
Study program: Computer Science, Specialization: General Computer Science
Supervisor: Mgr. Tomáš Urbanec
Page count: 19
Supplements: 1 CD/DVD
Thesis language: Czech

Anotace

Práce má dva cíle: 1. nastudovat principy ukládání a práce s daty v grafových databázích a získané poznatky sepsat do textu, který čtenáři poskytne dobrou představu o fungování, výhodách a nevýhodách tohoto přístupu k datům. 2. vybrat několik implementací grafových databází, nachystat ukázky jejich použití a pokusit se navrhnout experimenty ukazující silné a slabé stránky zvolených implementací.

Synopsis

Thesis has two goals: 1. study the principles of storing and working with data in graph databases and and write down the knowledge gained in a text that will give the reader a good idea of how it works, advantages and disadvantages of this approach to data. 2. select several implementations of graph databases, prepare examples of their use and try to devise experiments showing the strengths and weaknesses of the selected implementation.

Klíčová slova: grafové databáze; implementace grafových databází

Keywords: graph databases; implementation of graph databases

Děkuji, děkuji, děkuji.

Místopřísežně prohlašuji, že jsem celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

datum odevzdání práce

podpis autora

Obsah

1	Grafové databáze	7
1.1	Základní pojmy z teorie grafů	7
1.2	Modelování grafových dat	8
1.2.1	Definice problému	8
1.2.2	Konceptuální datový model	10
1.2.3	Logický datový model	12
1.2.4	Testování modelu	15
2	Implementace grafových databází	15
2.1	Neo4j	16
2.2	OrientDB	16
2.3	Dotazovací jazyky v grafových databázích	16
2.3.1	Gremlin	16
2.3.2	Cypher	16
2.3.3	GQL - Graph Query Language	16
	Závěr	17
	Conclusions	18
	Seznam zkratk	19

Seznam tabulek

1 Grafové databáze

Grafové databáze, stejně jako jiné databáze (například relační, NoSQL) představují systém pro uchovávání a zpracovávání dat. Jsou, jak již název napovídá, postaveny na jednom z oborů diskrétní matematiky jménem teorie grafů, podobně jako jsou relační databáze postaveny na pojmu relace. Pracují tedy s daty v podobě grafu. Díky této struktuře mají skvělé vlastnosti pro aplikace, které ve větší míře pracují se vztahy mezi jednotlivými entitami.

1.1 Základní pojmy z teorie grafů

V této části jsou uvedeny základní definice z teorie grafů používané v textu.

Definice 1 (Graf)

Graf G je uspořádaná dvojice $\langle V, E \rangle$, kde V označuje množinu vrcholů a $E \subseteq V \times V$ označuje množinu hran. Každá hrana je určena dvěma vrcholy z V , které spojuje.

Grafy se dělí na orientované a neorientované. Hrana v neorientovaném grafu mezi vrcholy u a v se nazývá neorientovaná a zapisuje se pomocí množiny obsahující dvojici vrcholů, tedy u, v . V případě orientovaného grafu se hrana nazývá orientovaná a pro zápis se používá uspořádaná dvojice. Orientovaná hrana (u, v) začíná ve vrcholu u a končí ve vrcholu v . Grafy se nejčastěji znázorňují pomocí kroužků, které představují vrcholy, a úseček mezi nimi, sloužící pro reprezentaci hran. Pokud se jedná o orientovaný graf místo úseček používáme šipky. Pro hrany používáme označení násobné, pokud existuje mezi dvěma vrcholy hran více. O hraně začínající i končící ve stejném vrcholu mluvíme jako o smyčce.

Definice 2 (Podgraf)

Podgraf grafu $G = \langle V, E \rangle$ je graf $H = \langle V1, E1 \rangle$, kde $V1 \subseteq V$ a $E1 \subseteq E$, takže každá hrana z $E1$ má oba vrcholy ve $V1$.

Definice 3 (Stupeň vrcholu)

Stupněm vrcholu u v grafu $G = \langle V, E \rangle$ se rozumí počet hran dotýkajících se u . Stupeň vrcholu u lze v orientovaném grafu dále rozdělit podle hran vcházejících a vycházejících na vstupní a výstupní.

Definice 4 (Cesta)

Cesta je sekvence vrcholů a hran, kde se vrcholy ani hrany neopakují. Cesta délky n má $n + 1$ vrcholů.

Definice 5 (Kružnice)

Kružnice je cesta, kde počáteční a koncový vrchol je stejný.

Definice 6 (Vzdálenost)

Vzdálenost z vrcholu u do vrcholu v lze definovat jako délku nejkratší cesty z u do v .

Definice 7 (Strom)

Strom je souvislý graf $G = \langle V, E \rangle$ neobsahující kružnice.

1.2 Modelování grafových dat

Modelování dat je proces, který překládá entity reálného světa a vztahy mezi nimi do jejich softwarové reprezentace. Uživatel popisuje doménu jako graf skládající se z vrcholů a hran pomocí vlastností a popisků. Každý výsledný model je unikátní. U každého záleží na případech použití a otázkách, které mají podobu dotazů, které budou na databázi kladeny. Dodržení základních principů modelování dat je důležité pro výslednou efektivitu. Když je graf přizpůsoben dotazům, dokáže velmi rychle nalézt potřebnou odpověď. Naopak špatně zvolený model grafu může ve výsledku aplikaci ublížit a být překážkou. Další důležitou otázkou je budoucnost vyvíjené aplikace. Model by měl být připravený na změny, které po aplikaci budou její koncoví uživatelé požadovat. Správnému modelu je proto dobré věnovat dostatečné množství času a dodržovat obecná pravidla.

Postup modelování se dá rozdělit do čtyř fází:

1. Definice problému
2. Konceptuální datový model
3. Logický datový model
4. Testování modelu

V této sekci se budeme snažit dobře navrhnout grafový model dat. Zkusíme si to na příkladu sociální platformy pro nadšence cestování.

1.2.1 Definice problému

V první fázi bychom měli zvážit co všechno by měla aplikace umět. Výsledkem by měl být její, co nejpřesnější, slovní popis. Pomoci nám může těchto pár otázek.

Co od naší aplikace uživatel očekává? Aplikace je sociální síť, proto by měla jistě umožnit se propojit s ostatními uživateli aplikace. Uživatelům by na základě jejich přátel, oblíbených aktivit a hodnocení destinací měly přicházet tipy na výlet. Aplikace by měla umožnit zaznamenat, že někteří uživatelé podnikli cestu společně.

Jaké typy informací naše aplikace potřebuje uchovávat, aby měla danou funkcionalitu?

- Všechny základní informace o **uživateli**, aby se s nimi ostatní mohli spojit - jejich jméno, příjmení, unikátní identifikátor, oblíbené cestovatelské aktivity.

- Informace o jednotlivých **destinacích** - jejich lokace, název, unikátní identifikátor, cestovatelské aktivity v rámci destinace.
- Uživateli je umožněno psát **hodnocení** na jednotlivé destinace.
- Chtěli bychom také umožnit **seskupení uživatelů**, když jedou do dané destinace společně.

V reálném scénáři by projekt pravděpodobně zahrnoval mnoho dalších atributů, ale pro tuto ukázkovou aplikaci to ponecháme omezené. Pro zjednodušení se vyhneme také souřadnicovým informacím a vystačíme si s informací v rámci jakého státu se daná destinace nachází.

Aplikace by tedy měla zacházet s pojmy jako jsou uživatel, destinace, hodnocení, aktivity. Jak spolu tyto pojmy vzájemně souvisí v rámci aplikace?

- Uživatel **sleduje** jiného uživatele.
- Uživatel **se zúčastnil** cesty **do** destinace.
- Uživateli se **líbí** cestovatelská aktivita.
- Uživatel **napsal** recenzi **o** cestě.
- Destinace **nabízí** cestovatelskou aktivitu.

Na jaké otázky potřebují uživatelé najít odpověď?

- Jaké všechny uživatele sleduji? A koho sledují oni?
- Najdi 3 nejlépe hodnocené destinace, které jsem nenavštívil, nabízející konkrétní cestovní aktivity?
- Které destinace navštívil uživatel X. Kolikrát je navštívil?
- Najdi mi všechny uživatele v rámci skupiny, s kterou cestuje uživatel X do destinace Y.
- Najdi X destinací nabízející aktivity, které se uživateli líbí.
- Pro konkrétní destinaci vrať měsíc v roce v kterém je nejlépe hodnocená.

V reálném scénáři by otázek bylo mnohem více.

1.2.2 Konceptuální datový model

Dalším krokem je vyvinout konceptuální model. Výstupem by měl být přehledný diagram popisující důmyslně promyšlené schéma naší aplikace z business pohledu. Vyvarujeme se v něm pro nevyvojáře zbytečným detailům a zaměříme se pouze na to, jak data různých typů spolu v rámci vyvíjené aplikace korelují. Při vyvíjení datového modelu lidé často kreslí na tabuli, jak jsou data v rámci projektu propojena s dalšími daty, proto se tato fáze v různých literaturách nazývá whiteboard model. Vyžaduje identifikování a seskupení všech entit a vztahů, které se v naší aplikaci vyskytují. Začneme entitami. Entity většinou představují objekty, jako například uživatel, účet, destinace, které jsou v databázi uloženy a propojeny vzájemnými vztahy. Je třeba si rozmyslet, které entity lze seskupit. Pokud například naše aplikace zahrnuje uživatele, zaměstnance a administrátora je užitečné tuto skupinu entit sloučit do jedné. Jedním z postupů jak z našich poznámek z minulé fáze vytáhnout entity, je dívat se po podstatných jménech.

V naší aplikaci můžeme rozpoznat 6 různých entit:

- Uživatel
- Cesta
- Destinace
- Aktivita
- Stát
- Recenze

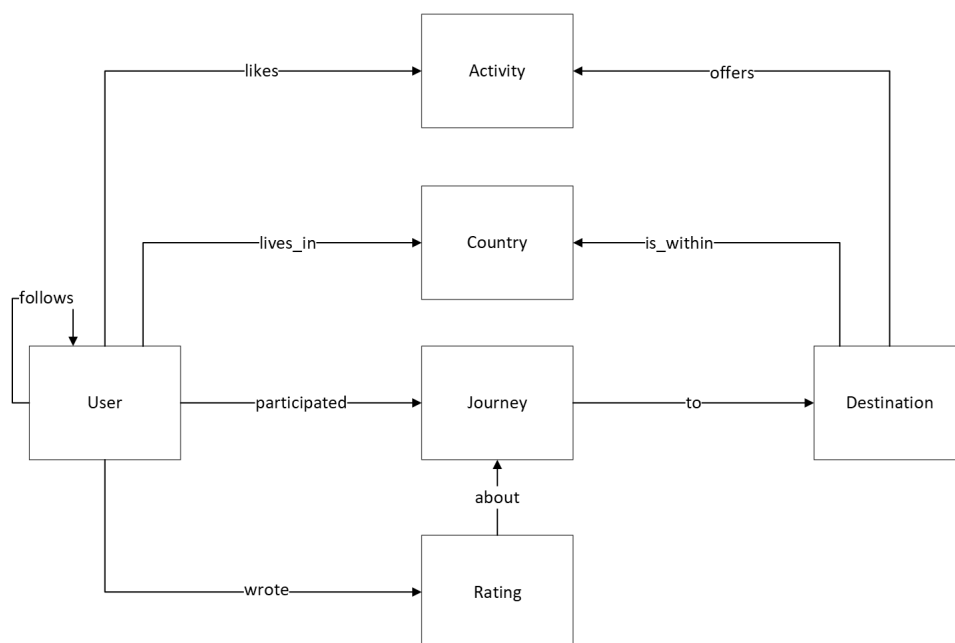
Naše tabule tedy má nyní podobu, kterou vystihuje [Obrázek 1](#).

Nyní následuje propojení entit pomocí vztahů. Podobně jako entity lze zejména nalézt zaměřením se na podstatná jména, tak většina vztahů můžeme identifikovat pomocí sloves. Zde bychom však měli být velmi opatrní. Ve slovese se často ukrývá podstatné jméno. V budoucím vývoji aplikace by pak mohlo dojít k problémům při rozšiřování funkcionality aplikace. Dobrým příkladem je sloveso recenzovat. V případě, že bychom při modelování zvolili větu "Uživatel recenzoval X." místo "Uživatel napsal recenzi o X." mohli bychom narazit na problém, že nemáme entitu představující recenzi, na kterou bychom mohli odkazovat. Snadno si jde domyslet komentování nebo jiný způsob reagování na recenze. Podobně pokud bychom v naší aplikaci upřednostnili větu "Uživatel navštívil destinaci." před větou "Uživatel podnikl cestu do destinace.", nemohli bychom pak na danou cestu odkazovat. Takové chyby obecně vedou v budoucnosti k migrování databáze, což je velmi obtížný a zdoluhavý proces přesouvání dat z jedné databáze do druhé, kterému je lepší se vyhnout.

Po důkladném rozmyšlení naše tabule má obsah, jako [Obrázek 2](#).



Obrázek 1: Entity vyskytující se v aplikaci sociální síť pro cestovní nadšence



Obrázek 2: Konceptuální datový model pro sociální síť pro cestovní nadšence

1.2.3 Logický datový model

Pokud jsme s diagramem z předchozí fáze spokojeni, můžeme konečně přistoupit ke kroku, ve kterém definujeme schéma, které bude v databázi. V případě relačního modelování je konceptuální model přeformátován, aby mohl být ukládán do tabulek. V případě grafového modelování toto není potřeba. Stačí identifikovat vrcholy a hrany a přiřadit jim příslušné popisky a vlastnosti. Výstupem tedy bude opět diagram, který ale bude mnohem detailnější. Bude se zabývat detaily, kterými se zabývají vývojáři. Bude definovat tvar, ve kterém budou data v databázi uložena a podle něj budeme moci klást na databázi dotazy.

Pro vytvoření grafového modelu jsou potřeba tyto 3 kroky:

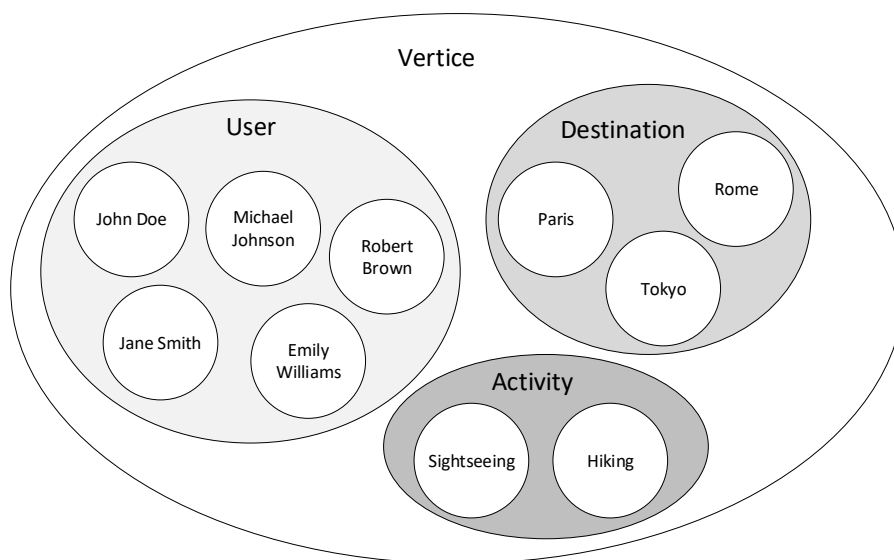
1. Převést entity na vrcholy
2. Převést relace na hrany
3. Najít a přiřadit vlastnosti vrcholům a hranám

První jsou na řadě vrcholy. Podstatnou část práce již máme za sebou, neboť můžeme využít konceptuální model a namapovat entity na vrcholy. Zbývá jen jeden netriviální úkol - každému vrcholu přiřadit krátký, přesný a výstižný popis. Popisek uzlu určuje, do které skupiny nebo kategorie uzlů patří. Všechny uzly se stejným popiskem pak spadají do stejné kategorie. Pro lepší pochopení je zde Obrázek 3. Tyto popisky pak můžeme používat v dotazech na databázi a pracovat tak s různými skupinami uzlů a dosáhnout tak lepšího výkonu při dotazování.

Při vymýšlení popisků musíme mít jako vždy na paměti budoucí rozšiřitelnost a fakt, že s databází může pracovat jiný vývojář. Dobře zvolený popisek je tedy velmi žádaný. Důležité je též dbát na pojmenovovací konvenci. V našem příkladu je zvolena konvence doporučená v dokumentaci databáze Neo4j a tedy pro popisky vrcholů budeme používat CamelCase notaci, pro popisky hran UPPER_CASE notaci s použitím podtržítka pro oddělení slov a pro vlastnosti snake_case notaci. Dobrým zvykem je zvolit popisek, který bude v jednotném čísle, neboť každý uzel odkazuje na jednu instanci daného popisku. Při přiřazování popisku je dobré myslet na jeho co největší obecnost, protože v budoucnu bychom mohli potřebovat reprezentovat jiný podobný koncept a tedy je užitečné volit co nejobecnější popisky.

Povolený počet popisků uzlu se u různých grafových databází liší. Bezpečným přístupem je každému vrcholu přiřadit jeden popisek. To je například přístup Apache TinkerPop. Často se ale také hodí mít popisků víc a pomocí nich umět vyjádřit dědičnost. Pokud bychom spravovali zaměstnance v nějaké firmě, hodilo by se někdy k popisku zaměstnanec mít popisek ředitel, nadřízený apod. Tyto uzly pak spadají do více skupin uzlů naráz. Proto například Neo4j nebo Amazon Neptune umožňují uzlům přiřadit žádný, jeden ale i více popisků.

Při hledání hran opět využijeme důsledně připraveného konceptuálního modelu. Znovu se zamyslíme nad pojmenováním hran, které by opět mělo být obecné



Obrázek 3: Popisek uzlu definuje do které kategorie uzel spadá

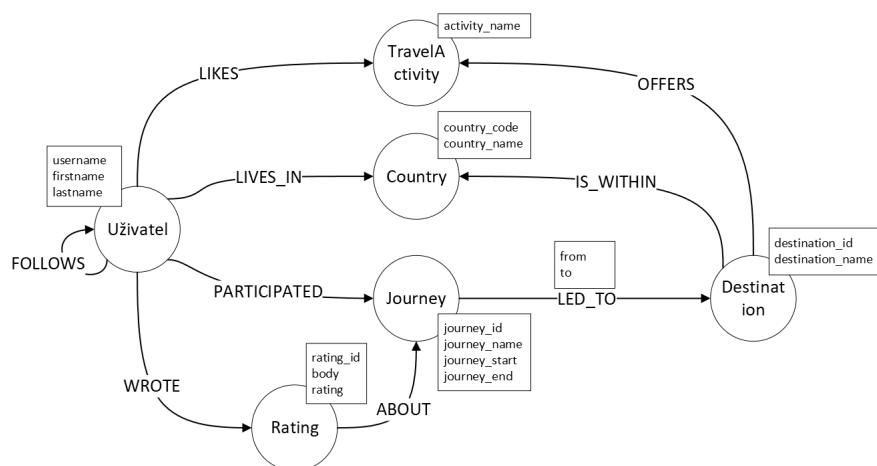
a výstižné. Navíc bychom se měli držet jedné dobré zásady. Přechzení posloupnosti vrcholu, hrany z něj vycházející a vrcholu do nějž hrana vstupuje by nám mělo dávat smysluplnou větu. Tedy například “Uživatel napsal hodnocení.“, nebo “Uživatel sleduje uživatele.“. To nás přivádí k orientaci hran. Předchozí rada nám tedy specifikuje orientaci hrany, neboť obrácením hrany bychom dostali věty “Hodnocení napsalo uživatele.“, nebo “Uživatel sleduje uživatele.“. V druhém případě se nám smysl věty nezměnil, jelikož se jedná o smyčku jdoucí z jednoho vrcholu do toho samého vrcholu. V prvním případě, kdy bychom požadovali opačnou orientaci hran mezi vrcholy “Uživatel“ a “Hodnocení“, bychom museli přistoupit ke změně popisku hrany, neboť by věta postrádala smysl. Tedy například hraně “napsal“ bychom změnili popisek na “bylo napsáno“ a pak by věta “Hodnocení bylo napsáno uživatelem.“ dávala smysl.

Než přejdeme k vlastnostem, zbývá ještě určit jedinečnost hran. Jedinečnost lze definovat jako počet hran se stejným popiskem mezi dvěma vrcholy. Tyto vrcholy mohou a nemusí mít stejný popisek. Jedinečnost hran se dělí na dva typy jednoduchou a mnohonásobnou. Jednoduchá odkazuje na žádnou, nebo právě jednu hranu zatímco vícenásobná povoluje hran se stejným popiskem mezi dvěma vrcholy více. Co se týká jedinečnosti hran v naší aplikaci tak všechny hrany, až na hranu mezi Cestou a Destinací budou mít jedinečnost jednoduchou. Mezi Uživatelem a Cestovní aktivitou vztah “má rád“ buď je nebo není a to lze vyjádřit pomocí existence jedné hrany mezi těmito dvěma uzly. Totéž platí pro ostatní hrany až na hranu s popiskem “vede do“. Můžeme si představit, že v rámci cesty uživatel do dané destinace zavítá vícekrát a s touto možností by náš model měl počítat. Tedy tato hrana bude mít vícenásobnou jedinečnost. Tyto hrany pak od sebe rozlišíme pomocí vlastností, které budou určovat období, kdy se daný uživatel v dané destinaci nacházel.

Jak zjistíme, že jsme zvolili špatnou jedinečnost hran? Špatná volba se projeví v dotazech jedním z těchto způsobů:

- Databáze vrátí příliš málo dat. Tato chyba nastane pokud zvolíme jednoduchou jedinečnost hran tam, kde by měla být zvolena vícenásobná jedinečnost.
- Databáze vrátí duplikativní data. Tato chyba nastane pokud zvolíme vícenásobnou jedinečnost hran tam, kde by měla být zvolena jednoduchá jedinečnost.
- Slabý výkon dotazů na databázi. Tato chyba nastane pokud zvolíme vícenásobnou jedinečnost hran tam, kde by měla být zvolena jednoduchá jedinečnost. Každá hrana stejného popisku mezi dvěma vrcholy navíc znamená rozvětvení v případě průchodu grafem a zvyšuje tedy výslednou časovou složitost dotazu.

U některých implementací grafových databází, jako například DataStax Graph nebo JanusGraph, lze tuto jedinečnost vynutit jako součást při definování schématu. U implementací bez schématického přístupu, jako je Neo4j, tuto jedinečnost lze ošetřit pouze v rámci aplikace.



Obrázek 4: Logický datový model pro sociální síť pro cestovní nadšence

Poslední zastávkou při konstruování logického modelu jsou vlastnosti. Základní kameny grafu vrcholy a hrany máme, ale chybí nám způsob jak rozlišit uzly se stejným popisem. K tomu slouží vlastnosti. V grafových databázích mohou mít vlastnosti jak hrany tak vrcholy, neboť hrany jsou pro nás stejně významné jak vrcholy. Vlastnosti v grafové databázi jsou páry klíč-hodnota a popisují konkrétní atributy hrany, či vrcholu. Hodnota vlastnosti může být různého datového typu, například řetězec, číslo, pravdivostní hodnota nebo datový typ pro uchování datumu a času. Někdy též může být hodnota typu pole nebo seznam. Pomocí vlastností následně můžeme rozlišit jednotlivé uživatele pomocí jejich křestního jména, příjmení, uživatelské přezdívky apod. K dokončení logického modelu nám tedy zbývá najít a správně přiřadit vlastnosti konkrétním vrcholům a hranám, pojmenovat je a určit jejich datový typ. K jejich nalezení nám pomohou otázky, které budeme chtít aby nám databáze zodpověděla. Z nich můžeme jednotlivé vlastnosti identifikovat a přiřadit.

Náš výsledný datový model popisuje Obrázek 4.

1.2.4 Testování modelu

V budoucnu je možné přidat entitu cestovní kanceláře, která bude pořádat cesty do destinací a uživatelé po objednání budou do této cesty připojeni. To otevírá dveře dalším entitám například pro hodnocení cestovních kanceláří, nebo vztahům které se týká zaměstnanosti uživatelů u dané cestovky.

2 Implementace grafových databází

Existuje spousta implementací grafových databází. Nejznámějšími jsou Neo4j, OrientDB, MemoGraph, TigerGraph, InfiniteGraph...

2.1 Neo4j

Jedná se o nejrozšířenější open-source grafovou databázi. Implementována je v Javě. Nabízí vysoký výkon při práci s grafy. Používá Cypher jako dotazovací jazyk.

2.2 OrientDB

OrientDB je open-source NoSQL databázový systém implementovaný v Javě. Je to multi-model databáze - podporuje grafový, dokumentový, hodnota/klíč a objektový model...

2.3 Dotazovací jazyky v grafových databázích

2.3.1 Gremlin

2.3.2 Cypher

2.3.3 GQL - Graph Query Language

Závěr

Závěr práce v „českém“ jazyce.

Conclusions

Thesis conclusions in “English”.

